services\analysis\cwe\processor.py

```
"""
CWE Severity Processor
Main processor for CWE severity analysis and data aggregation
"""

# Efficient counting and nested dictionary structures for data analysis
from collections import Counter, defaultdict
# Type hints for function parameters and return values
from typing import Dict, List, Any
# Date operations for temporal analysis
from datetime import datetime
# Random number generation (imported but not currently used)
import random

# CWE catalog management for detailed weakness information
from .catalog import CWECatalogLoader
# Centralized CWE title mappings for consistent labeling
from .constants import CWE_TITLES


class CWESeverityProcessor:
"""Process CWE data for severity analysis"""

def __init__(self):
# Store reference to CWE titles for consistent labeling
self.cwe_titles = CWE_TITLES
# Initialize catalog loader for detailed CWE information
self.catalog_loader = CWECatalogLoader()

def get_cwe_severity_data(self, top_n: int = 10) -> Dict[str, Any]:
"""Get CWE severity analysis data - LAZY LOADING"""
print(f"[CWE Processor] Analyzing CWE severity data (top {top_n})")

# Collect vulnerability data from all available sources
all_cves = self._collect_cve_data()

# Return empty structure if no data available for analysis
if not all_cves:
print(f"[CWE Processor] No data found, returning empty structure")
return self._get_empty_cwe_data(top_n)

# Analyze CWE frequency and severity distributions
cwe_severity_matrix, cwe_totals = self._analyze_cwe_severity(all_cves)

# Get top N CWEs by total occurrence count
top_cwes = cwe_totals.most_common(top_n)

# Build chart-ready data structure for visualization
result = self._build_chart_data(top_cwes, cwe_severity_matrix, len(all_cves))

print(f"[CWE Processor] Processed {len(all_cves)} CVEs, found {len(top_cwes)} CWEs")
print(f"[CWE Processor] Top CWEs: {[cwe for cwe, count in top_cwes]}")

return result

def _collect_cve_data(self) -> List[Dict]:
"""Collect CVE data from all sources - LAZY LOADING"""
all_cves = []

# Use lazy imports to avoid circular import issues
try:
from services.data.api_client import api_client
from services.data.data_processor import historical_loader

# Get recent data from API (last 30 days)
try:
api_cves = api_client.get_cves_last_30_days()
all_cves.extend(api_cves)
print(f"[CWE Processor] Got {len(api_cves)} API CVEs")
except Exception as e:
print(f"[CWE Processor] Error getting API CVEs: {e}")
```

```
# Get last 2 years of historical data for trend analysis - ONE YEAR AT A TIME
try:
current_year = datetime.now().year

# Load years separately to avoid memory issues
for year in [current_year - 1, current_year]:
year_data = historical_loader.get_year_data(year)  # Lazy load
all_cves.extend(year_data)
print(f"[CWE Processor] Got {len(year_data)} CVEs for {year}")

except Exception as e:
print(f"[CWE Processor] Error getting historical CVEs: {e}")

except Exception as e:
print(f"[CWE Processor] Import error: {e}")

print(f"[CWE Processor] Total CVEs to analyze: {len(all_cves)}")
return all_cves

def _analyze_cwe_severity(self, all_cves: List[Dict]) -> tuple:
"""Analyze CWE severity distribution"""
# Nested Counter for severity distribution per CWE
cwe_severity_matrix = defaultdict(lambda: Counter())
# Overall CWE frequency counter
cwe_totals = Counter()

# Process each CVE for CWE and severity information
for cve in all_cves:
cwe = cve.get('CWE')

# Validate CWE format before processing
if cwe and cwe.startswith('CWE'):
severity = cve.get('Severity', 'UNKNOWN').upper()

# Count valid severities and track in matrix
if severity in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
cwe_severity_matrix[cwe][severity] += 1
cwe_totals[cwe] += 1
else:
# Handle unknown or invalid severities
cwe_severity_matrix[cwe]['UNKNOWN'] += 1
cwe_totals[cwe] += 1

return cwe_severity_matrix, cwe_totals

def _build_chart_data(self, top_cwes: List[tuple], cwe_severity_matrix: dict,
total_cves: int) -> Dict[str, Any]:
"""Build chart data structure"""
# Initialize parallel arrays for chart consumption
labels = []  # Human-readable CWE titles
indices = []  # CWE codes for reference
data = {  # Severity counts per CWE
'CRITICAL': [],
'HIGH': [],
'MEDIUM': [],
'LOW': [],
'UNKNOWN': []
}

# Process each top CWE for chart data
for cwe_code, total_count in top_cwes:
# Resolve CWE code to human-readable title
title = self.cwe_titles.get(cwe_code, cwe_code)
labels.append(title)
indices.append(cwe_code)

# Extract severity breakdown for this CWE
severity_counts = cwe_severity_matrix[cwe_code]
data['CRITICAL'].append(severity_counts.get('CRITICAL', 0))
data['HIGH'].append(severity_counts.get('HIGH', 0))
data['MEDIUM'].append(severity_counts.get('MEDIUM', 0))
data['LOW'].append(severity_counts.get('LOW', 0))
data['UNKNOWN'].append(severity_counts.get('UNKNOWN', 0))

# Return chart-ready data structure
return {
'labels': labels,  # For chart axis labels
```

```
'indices': indices,  # For data reference
'data': data,  # For chart series data
'total_cves': total_cves  # For context information
}

def _get_empty_cwe_data(self, top_n: int = 10) -> Dict[str, Any]:
"""Return empty data structure when no data is available"""
return {
'labels': [],
'indices': [],
'data': {
'CRITICAL': [],
'HIGH': [],
'MEDIUM': [],
'LOW': [],
'UNKNOWN': []
},
'total_cves': 0
}

def get_cwe_details(self) -> Dict[str, Dict[str, Any]]:
"""Get CWE details for the explorer"""
# Start with key CWEs and provide loading placeholders
cwe_details = {}
for cwe_code, title in self.cwe_titles.items():
cwe_details[cwe_code] = {
'name': title,
'description': f'Click to fetch detailed information about {cwe_code}',
'mitigations': ['Loading detailed information...'],  # Loading indicator
'examples': [],
'relationships': []
}

# Add detailed CWEs from XML catalog without overwriting key CWEs
catalog = self.catalog_loader.load_cwe_catalog()
for cwe_code, cwe_data in catalog.items():
if cwe_code not in cwe_details:
cwe_details[cwe_code] = cwe_data

return cwe_details

def get_key_cwes(self) -> List[str]:
"""Get list of key CWE codes"""
# Return list of curated CWE identifiers
return list(self.cwe_titles.keys())

def get_key_cwe_titles(self) -> Dict[str, str]:
"""Get key CWE titles mapping"""
# Return copy of title mapping for external use
return self.cwe_titles.copy()

def get_cwe_by_code(self, cwe_code: str) -> Dict[str, Any]:
"""Get CWE details by code"""
# Delegate to catalog loader for individual CWE lookup
return self.catalog_loader.get_cwe_by_code(cwe_code)
```